



Thank you to our Fabric February Friends!



bouvet twoday soprano





The Ultimate Medallion Load Showdown: Azure v Fabric

Abhi Jayanty & Brian Børnk



Abhi Jayanty

Data Engineer
Quorum

✉ abhi.jayanty@quorum.co.uk

 [/in/abhinav-j](https://www.linkedin.com/in/abhinav-j)



Brian Bønck

Founder & MVP
ProBI

✉ bbo@probi.dk

 [/in/brianbonk](https://www.linkedin.com/in/brianbonk)

 brianbonk.dk
probi.dk

Agenda

- Set the scene
- Intro to batch v streaming ingestion
- Battle time!
- Crown the winner?
- Recap and final thoughts





Why are we here?

Why are we here?

- There are multiple leading data platforms for both batch or streaming based ingestion of data
 - Fabric, Azure data stack, Databricks...
- Streaming allows for more than just IoT/telemetry data use cases
- But surely batch is still the way to go?





Quick introduction to batch ingestion

What is batch ingestion?

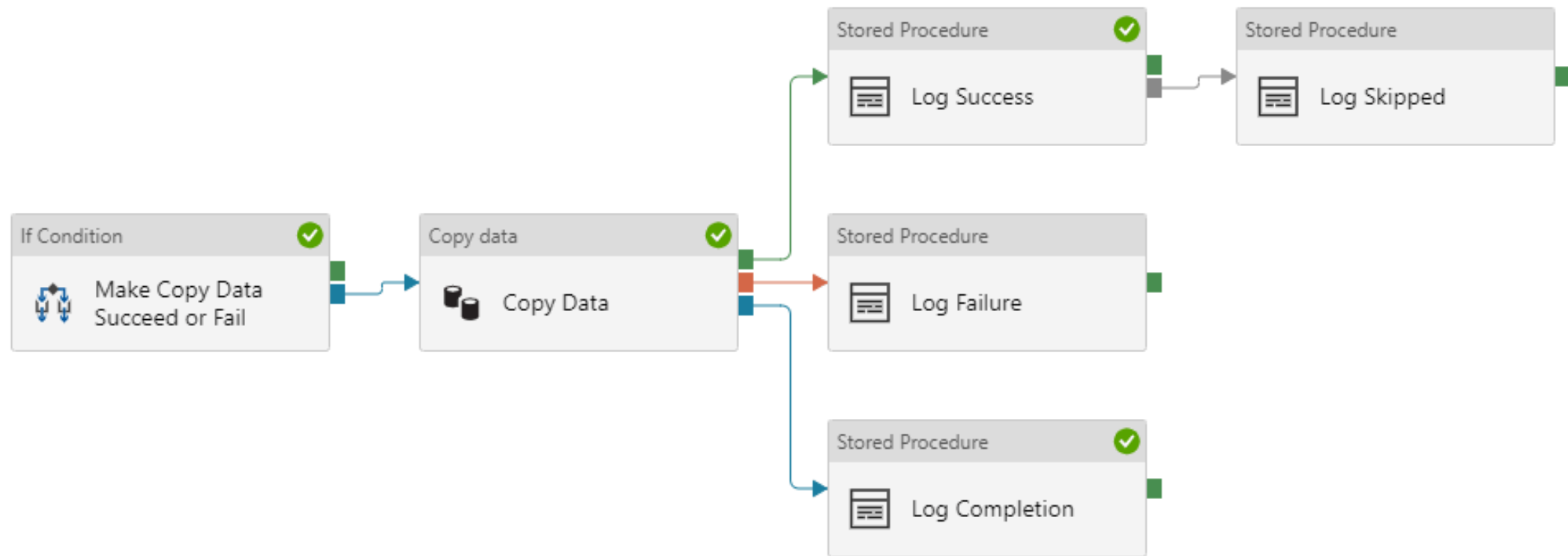
Batch data ingestion is the process of collecting and loading often large volumes of data at scheduled intervals — **hourly, nightly, or on demand**

Can be implemented with:

- Azure Data Factory (or Fabric pipelines...)
- Spark notebooks (schedule-based)



What is batch ingestion?





Quick introduction to streaming ingestion

What is streaming ingestion?

Streaming data ingestion captures and processes data continuously as events happen –
in seconds or even milliseconds

Can be implemented with:

- Fabric Real-Time Intelligence
- Azure Event Hubs/Stream Analytics



Fabric Real-Time Intelligence



Enterprise real-time data platforms

Azure Event Hubs

Azure Event Grid

Azure Stream Analytics

Azure Data Explorer

+



Self-service reporting and activation

OneLake

Data Activator

Power BI

=



Real-Time Intelligence in Microsoft Fabric

Fully integrated

No/low-code real-time SaaS

Data platform



Fabric Real-Time Intelligence



Eventstream

Native support for Azure Event/IoT Hubs and various 3rd party sources

No-code experience to transform and clean data in motion



Eventhouse

Logical collection of KQL databases



KQL DB



KQL DB



KQL DB

Optimised for time-series data



Real-Time Dashboards

Modular dashboards populated by 'tiles' powered by KQL queries

Continuous automatic refresh



Time to battle!



The Rules

- Same dataset
- Same 'medallion' enrichment
- Visualize the data at the end:
 - Power BI for batch
 - Fabric Real-Time Dashboards for streaming



Copy job

Choose data source
Select a connector. Then enter the connection information.

Choose data

Choose data destination

Choose copy job mode

Map to destination

Review + save

Home New OneLake catalog Azure Sample data

Search

New sources

SQL Server database Database

Dataverse Power Platform

SharePoint Online list Online services

Azure SQL database Azure

Folder File

OData Other

Oracle database Database

Odbc Other

View more →

OneLake catalog

View more →

All Recent Recommended

Name	Type	Owner	Refreshed	Location	Endorsement	Sensitivity
Eventhouse	Lakehouse	Brian Bønk	—	FabFeb2026	—	—
RetailAnalytics	Lakehouse	Brian Bønk	—	MatView with Graph...	—	—
HouseByTheLake	Lakehouse	?	—	Sanne KAB	—	—
Lakehouse	Lakehouse	Brian Bønk	—	Real Time demo	—	—
DennisOnsdag	Warehouse	?	—	Dennis KAB	—	—
DennisSidste	Warehouse	?	—	Dennis KAB	—	—



Home

Workspaces

Copilot

OneLake catalog

Monitor

Deployment pipelines

Real-Time

Workloads

FabFeb2026

My workspace

Brian Benk

Fabric

Lakehouse

Search

100

Home

Get data

New semantic model

Open notebook

Add to data agent

Manage materialized lake views (preview)

Manage OneLake security (preview)

Update all variables

Lakehouse

Share

A SQL analytics endpoint for SQL querying was created with this item.

Explorer

Lakehouse

Tables

dbo

Employees

Invoices

Products

Files

Get data in your lakehouse

Upload files

Upload data from your local machine.

Start with sample data

Automatically import tables filled with sample data.

New shortcut

Access data that resides in an external lake.

New Dataflow Gen2

Prep, clean, transform, and ingest data.

New pipeline

Ingest data at scale and schedule data workflows.



So who wins?

BATCH

STREAMING

Overall cost



Speed of ingestion



Consistency of batches and transactions



Downstream triggers and actions



Batch – Advantages

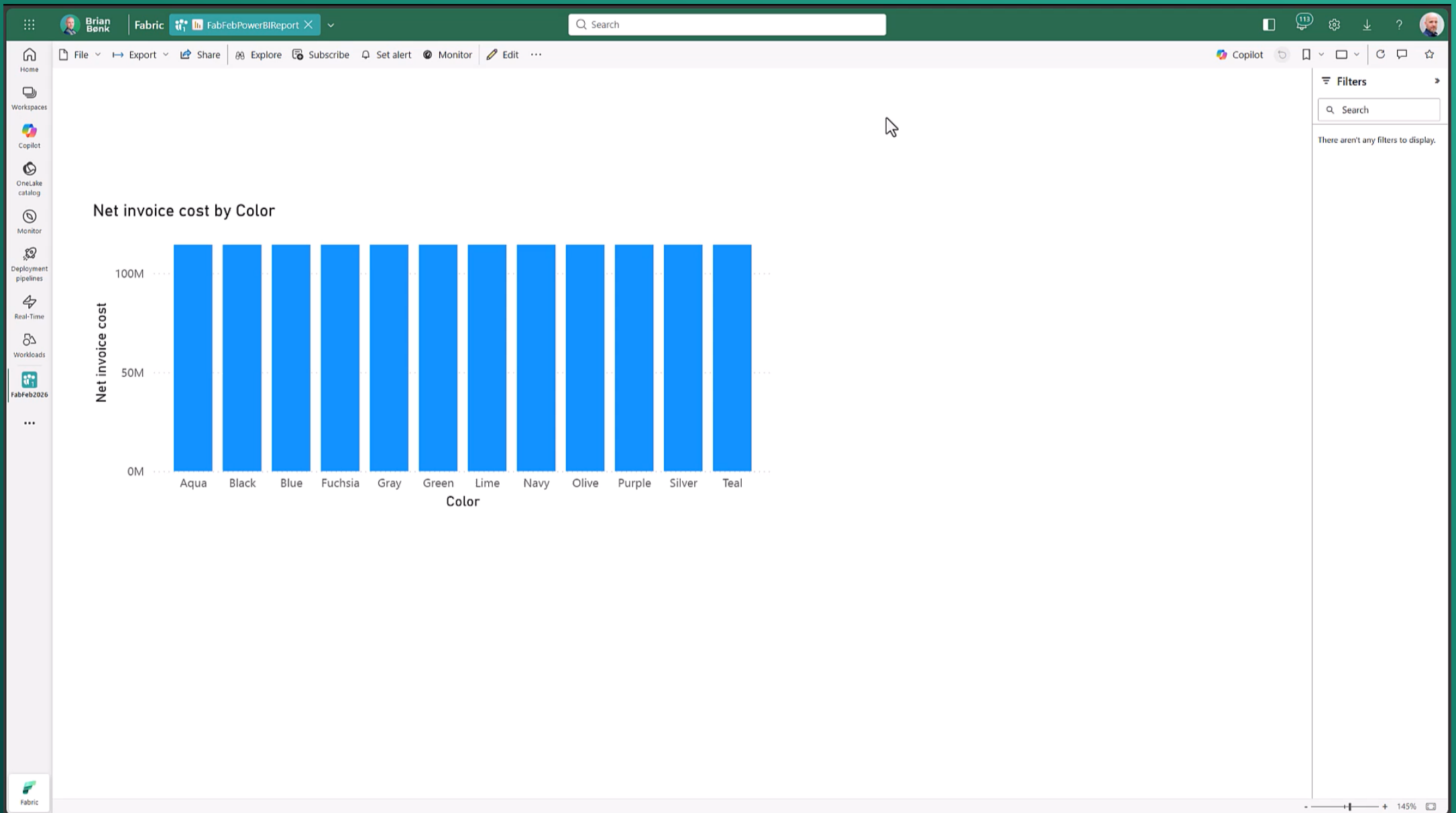
- Predictable
- Better for consistent, transactional data
- Can safely ingest batches of a large volume
- Cheap to run, especially in PaaS world



Batch – Considerations

- Need to create all connections:
 - Source to pipeline
 - Sub-pipeline to the next
 - Pipeline to destination
 - Destination to analysis or visualization layer
- Scheduling – if one pipeline fails or data lands late, then an entire re-run process is required
- Downstream triggers and refreshes not possible (unless you use PowerAutomate or API calls)





Streaming – Advantages

- Super-fast ingestion, data lands whenever it is available, no schedule needed
- Automatic downstream triggers and report refreshes (Fabric Activator)
- SaaS with RTI – time to development is quicker, all transformations can sit in ES or EH (no sub-pipeline mess)



Streaming – Considerations

- Cost – constant background processing can hurt capacity
- Requires better understanding of event-driven data
 - high volume, high duplication, inconsistent data = stronger cleansing
- Often could be 'overkill' for a dataset that does not constantly change – use case and scenario is very important





But wait!

We can still be friends 😊

Why not both in your data estate?

- RTI can still use Lakehouse as a destination in Fabric, so essentially becomes another option for ingestion alongside pipelines and notebooks
- ADF or other ETL tools can be very useful for orchestrating and batching out bulk data into messages for event-based destinations (service bus etc.)

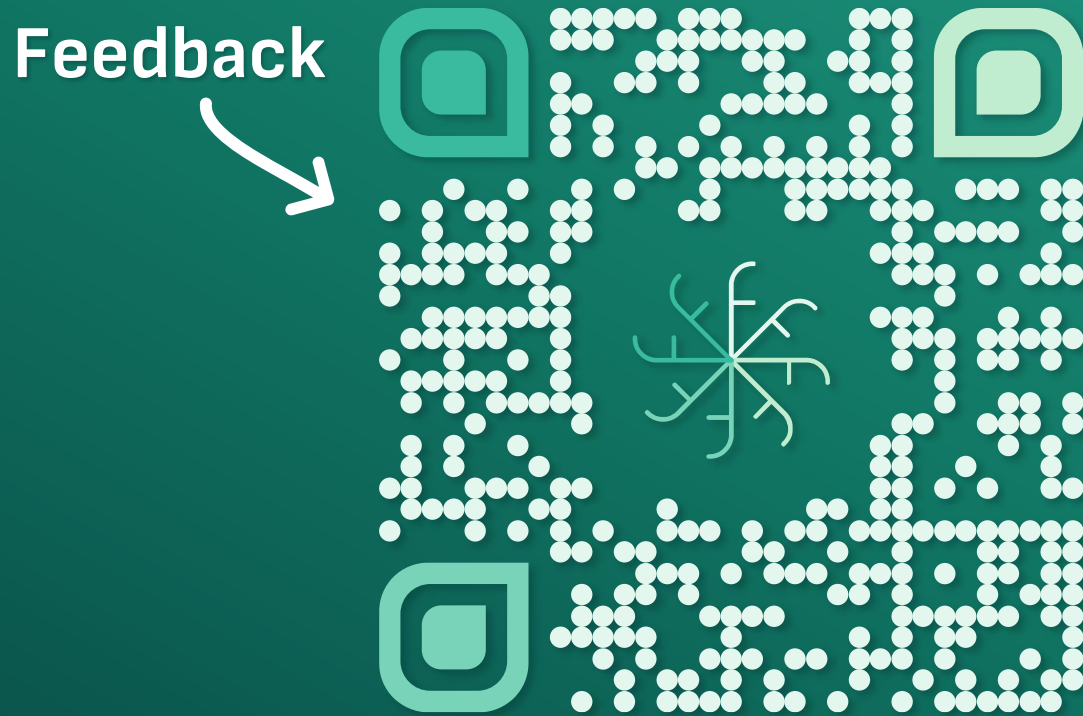


Links and Resources

- <https://brianbonk.dk/blog/worlds-fastest-medallion-architecture-load/>
- Azure Data Explorer — dataexplorer.azure.com
- Kusto Detective Agency
- KQL Reference guide — kql.how



Thank you!



fabfeb.app/feedback